AGLC Law Firm CRM: Code Examples & Implementation Patterns

Project: AGLC Law Firm CRM
Version: 1.0
Date: March 12, 2026
Purpose: Reference implementation patterns and code examples

Table of Contents

1. Adding a New Feature
2. Creating tRPC Procedures
3. Database Operations
4. Frontend Components
5. Error Handling
6. Testing Patterns
7. AI Agent Integration
8. Common Pitfalls

Adding a New Feature

Step-by-Step Example: Adding a "Notes" Feature to Cases

This example demonstrates the complete workflow for adding a new feature from database to UI.

Step 1: Define Database Schema

File: `drizzle/schema.ts`

```
import { int, mysqlTable, text, timestamp, varchar } from "drizzle-orm/mysql-core";

export const caseNotes = mysqlTable("caseNotes", {
  id: int("id").autoincrement().primaryKey(),
  caseId: int("caseId").notNull().references(() => cases.id),
  userId: int("userId").notNull().references(() => users.id),
  content: text("content").notNull(),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
  updatedAt: timestamp("updatedAt").defaultNow().onUpdateNow().notNull(),
});

export type CaseNote = typeof caseNotes.$inferSelect;
export type InsertCaseNote = typeof caseNotes.$inferInsert;
```

Key Points:

- Use `autoincrement()` for auto-incrementing IDs
- Use `references()` for foreign keys
- Use `defaultNow()` for timestamps
- Export both select and insert types

Step 2: Create Database Migration

```
pnpm db:push
```

This command automatically generates and applies migrations based on schema changes.

Step 3: Add Database Query Helpers

File: `server/db.ts`

```
import { eq, desc } from "drizzle-orm";
import { caseNotes, CaseNote, InsertCaseNote } from "../drizzle/schema";

export async function getCaseNotes(caseId: number): Promise<CaseNote[]> {
  const db = await getDb();
  if (!db) return [];

  return db
    .select()
    .from(caseNotes)
    .where(eq(caseNotes.caseId, caseId))
    .orderBy(desc(caseNotes.createdAt));
}

export async function createCaseNote(
  note: InsertCaseNote
): Promise<CaseNote | null> {
  const db = await getDb();
  if (!db) return null;

  const result = await db.insert(caseNotes).values(note);
  return getCaseNoteById(result.insertId);
}

export async function getCaseNoteById(id: number): Promise<CaseNote | null>
{
  const db = await getDb();
  if (!db) return null;

  const result = await db
    .select()
    .from(caseNotes)
    .where(eq(caseNotes.id, id))
    .limit(1);

  return result.length > 0 ? result[0] : null;
}

export async function updateCaseNote(
  id: number,
  updates: Partial<InsertCaseNote>
): Promise<CaseNote | null> {
  const db = await getDb();
  if (!db) return null;
```

```
  await db.update(caseNotes).set(updates).where(eq(caseNotes.id, id));
  return getCaseNoteById(id);
}

export async function deleteCaseNote(id: number): Promise<boolean> {
  const db = await getDb();
  if (!db) return false;

  await db.delete(caseNotes).where(eq(caseNotes.id, id));
  return true;
}
```

Best Practices:

- Always check if database is available
- Return typed results
- Use helper functions for common queries
- Handle errors gracefully

Step 4: Add tRPC Procedures

File: `server/routers.ts`

```
import { z } from "zod";
import { protectedProcedure, router } from "./_core/trpc";
import * as db from "./db";

export const appRouter = router({
  // ... existing routers ...

  caseNotes: router({
    list: protectedProcedure
      .input(z.object({ caseId: z.number() }))
      .query(async ({ input }) => {
        return db.getCaseNotes(input.caseId);
      }),

    create: protectedProcedure
      .input(z.object({
        caseId: z.number(),
        content: z.string().min(1).max(5000),
      }))
      .mutation(async ({ ctx, input }) => {
        return db.createCaseNote({
          caseId: input.caseId,
          userId: ctx.user.id,
          content: input.content,
        });
      }),

    update: protectedProcedure
      .input(z.object({
        id: z.number(),
        content: z.string().min(1).max(5000),
      }))
      .mutation(async ({ input }) => {
        return db.updateCaseNote(input.id, {
          content: input.content,
        });
      }),

    delete: protectedProcedure
      .input(z.object({ id: z.number() }))
      .mutation(async ({ input }) => {
        return db.deleteCaseNote(input.id);
      }),
```

```
  }),
});
```

Key Points:

- Use `protectedProcedure` for authenticated endpoints
- Use Zod for input validation
- Use `query` for read operations
- Use `mutation` for write operations
- Return typed results

Step 5: Create Frontend Component

File: `client/src/components/CaseNotes.tsx`

```
import { useState } from "react";
import { trpc } from "@/lib/trpc";
import { Button } from "@/components/ui/button";
import { Textarea } from "@/components/ui/textarea";
import { Card } from "@/components/ui/card";
import { Loader2, Trash2 } from "lucide-react";
import { toast } from "sonner";

interface CaseNotesProps {
  caseId: number;
}

export function CaseNotes({ caseId }: CaseNotesProps) {
  const [newNote, setNewNote] = useState("");

  // Query: Get all notes for this case
  const { data: notes, isLoading, refetch } = trpc.caseNotes.list.useQuery(
    { caseId },
    { enabled: !!caseId }
  );

  // Mutation: Create a new note
  const createMutation = trpc.caseNotes.create.useMutation({
    onSuccess: () => {
      setNewNote("");
      refetch();
      toast.success("Note added successfully");
    },
    onError: (error) => {
      toast.error(error.message || "Failed to add note");
    },
  });

  // Mutation: Delete a note
  const deleteMutation = trpc.caseNotes.delete.useMutation({
    onSuccess: () => {
      refetch();
      toast.success("Note deleted successfully");
    },
    onError: (error) => {
      toast.error(error.message || "Failed to delete note");
    },
  });

  const handleAddNote = async () => {
```

```
    if (!newNote.trim()) {
      toast.error("Note cannot be empty");
      return;
    }

    await createMutation.mutateAsync({
      caseId,
      content: newNote,
    });
  };

  const handleDeleteNote = async (id: number) => {
    if (confirm("Are you sure you want to delete this note?")) {
      await deleteMutation.mutateAsync({ id });
    }
  };

  return (
    <div className="space-y-4">
      <h3 className="text-lg font-semibold">Case Notes</h3>

      {/* Add Note Form */}
      <Card className="p-4">
        <Textarea
          placeholder="Add a note..."
          value={newNote}
          onChange={(e) => setNewNote(e.target.value)}
          rows={3}
        />
        <Button
          onClick={handleAddNote}
          disabled={createMutation.isPending || !newNote.trim()}
          className="mt-2"
        >
          {createMutation.isPending ? (
            <>
              <Loader2 className="mr-2 h-4 w-4 animate-spin" />
              Adding...
            </>
          ) : (
            "Add Note"
          )}
        </Button>
      </Card>

      {/* Notes List */}
```

```
      {isLoading ? (
        <div className="flex justify-center">
          <Loader2 className="h-6 w-6 animate-spin" />
        </div>
      ) : notes && notes.length > 0 ? (
        <div className="space-y-2">
          {notes.map((note) => (
            <Card key={note.id} className="p-4">
              <div className="flex justify-between items-start">
                <div className="flex-1">
                  <p className="text-sm text-gray-600">
                    {new Date(note.createdAt).toLocaleDateString()}
                  </p>
                  <p className="mt-1">{note.content}</p>
                </div>
                <Button
                  variant="ghost"
                  size="sm"
                  onClick={() => handleDeleteNote(note.id)}
                  disabled={deleteMutation.isPending}
                >
                  <Trash2 className="h-4 w-4" />
                </Button>
              </div>
            </Card>
          ))}
        </div>
      ) : (
        <p className="text-gray-500">No notes yet</p>
      )}
    </div>
  );
}
```

Best Practices:

- Use `useQuery` for data fetching
- Use `useMutation` for data mutations
- Show loading states
- Handle errors with toast notifications
- Refetch data after mutations
- Use optimistic updates when appropriate

Step 6: Add Component to Page

File: `client/src/pages/CaseDetail.tsx`

```
import { CaseNotes } from "@/components/CaseNotes";

export default function CaseDetail() {
  const { caseId } = useParams();

  return (
    <div className="space-y-6">
      {/* ... other case details ... */}

      <CaseNotes caseId={parseInt(caseId)} />
    </div>
  );
}
```

Step 7: Write Tests

File: `server/caseNotes.test.ts`

```
import { describe, it, expect } from "vitest";
import { appRouter } from "./routers";
import type { TrpcContext } from "./_core/context";

function createTestContext(): TrpcContext {
  return {
    user: {
      id: 1,
      openId: "test-user",
      email: "test@example.com",
      name: "Test User",
      loginMethod: "test",
      role: "user",
      createdAt: new Date(),
      updatedAt: new Date(),
      lastSignedIn: new Date(),
    },
    req: { protocol: "https", headers: {} } as any,
    res: {} as any,
  };
}

describe("caseNotes", () => {
  it("should create a case note", async () => {
    const ctx = createTestContext();
    const caller = appRouter.createCaller(ctx);

    const result = await caller.caseNotes.create({
      caseId: 1,
      content: "Test note",
    });

    expect(result).toBeDefined();
    expect(result?.content).toBe("Test note");
  });

  it("should list case notes", async () => {
    const ctx = createTestContext();
    const caller = appRouter.createCaller(ctx);

    const result = await caller.caseNotes.list({ caseId: 1 });

    expect(Array.isArray(result)).toBe(true);
  });
```

```
  it("should reject empty notes", async () => {
    const ctx = createTestContext();
    const caller = appRouter.createCaller(ctx);

    expect(async () => {
      await caller.caseNotes.create({
        caseId: 1,
        content: "",
      });
    }).rejects.toThrow();
  });
});
```

Creating tRPC Procedures

Query Procedure Example

```
// Read-only operation
export const appRouter = router({
  cases: router({
    list: protectedProcedure
      .input(z.object({
        status: z.enum(["Active", "Completed"]).optional(),
        limit: z.number().default(10),
        offset: z.number().default(0),
      }))
      .query(async ({ ctx, input }) => {
        // Only return cases assigned to the current user
        return db.getCases({
          userId: ctx.user.id,
          status: input.status,
          limit: input.limit,
          offset: input.offset,
        });
      }),
  }),
});
```

Mutation Procedure Example

```
// Write operation
export const appRouter = router({
  cases: router({
    create: protectedProcedure
      .input(z.object({
        clientId: z.number(),
        caseType: z.enum(["Corporate", "RealEstate", "Litigation"]),
        deadline: z.date(),
      }))
      .mutation(async ({ ctx, input }) => {
        // Create case and assign to current user
        return db.createCase({
          clientId: input.clientId,
          caseType: input.caseType,
          deadline: input.deadline,
          assignedTeam: [ctx.user.id],
        });
      }),
  }),
});
```

Admin-Only Procedure Example

```
// Restrict to admins only
const adminProcedure = protectedProcedure.use(({ ctx, next }) => {
  if (ctx.user.role !== "admin") {
    throw new TRPCError({ code: "FORBIDDEN" });
  }
  return next({ ctx });
});

export const appRouter = router({
  admin: router({
    getAnalytics: adminProcedure
      .input(z.object({ dateRange: z.enum(["week", "month", "year"]) }))
      .query(async ({ input }) => {
        return db.getAnalytics(input.dateRange);
      }),
  }),
});
```

Database Operations

Query Examples

```
// Get single record
const caseData = await db.select()
  .from(cases)
  .where(eq(cases.id, caseId))
  .limit(1);

// Get multiple records with filtering
const activeCases = await db.select()
  .from(cases)
  .where(eq(cases.status, "Active"))
  .orderBy(desc(cases.createdAt))
  .limit(10);

// Join tables
const casesWithClients = await db.select()
  .from(cases)
  .leftJoin(clients, eq(cases.clientId, clients.id))
  .where(eq(cases.status, "Active"));

// Aggregate query
const taskStats = await db.select({
  status: tasks.status,
  count: count(),
})
  .from(tasks)
  .groupBy(tasks.status);
```

Insert Examples

```
// Insert single record
await db.insert(cases).values({
  clientId: 1,
  caseType: "Corporate",
  status: "Intake",
});

// Insert multiple records
await db.insert(tasks).values([
  { phaseId: 1, title: "Task 1", priority: "High" },
  { phaseId: 1, title: "Task 2", priority: "Medium" },
]);
```

Update Examples

```
// Update single record
await db.update(cases)
  .set({ status: "Active" })
  .where(eq(cases.id, caseId));

// Update with conditions
await db.update(tasks)
  .set({ status: "Completed" })
  .where(and(
    eq(tasks.phaseId, phaseId),
    gte(tasks.deadline, new Date())
  ));
```

Delete Examples

```
// Delete single record
await db.delete(cases)
  .where(eq(cases.id, caseId));

// Delete with conditions
await db.delete(tasks)
  .where(and(
    eq(tasks.status, "Blocked"),
    lt(tasks.deadline, new Date())
  ));
```

Frontend Components

Form Component Example

```
import { useForm } from "react-hook-form";
import { zodResolver } from "@hookform/resolvers/zod";
import { z } from "zod";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { toast } from "sonner";

const caseFormSchema = z.object({
  clientId: z.number().min(1, "Client is required"),
  caseType: z.enum(["Corporate", "RealEstate", "Litigation"]),
  deadline: z.date(),
});

type CaseFormData = z.infer<typeof caseFormSchema>;

export function CaseForm() {
  const { register, handleSubmit, formState: { errors } } =
useForm<CaseFormData>({
    resolver: zodResolver(caseFormSchema),
  });

  const createMutation = trpc.cases.create.useMutation({
    onSuccess: () => {
      toast.success("Case created successfully");
    },
  });

  const onSubmit = async (data: CaseFormData) => {
    await createMutation.mutateAsync(data);
  };

  return (
    <form onSubmit={handleSubmit(onSubmit)} className="space-y-4">
      <div>
        <label>Client</label>
        <Input {...register("clientId", { valueAsNumber: true })} />
        {errors.clientId && <span className="text-red-500">
{errors.clientId.message}</span>}
      </div>
```

```
      <div>
        <label>Case Type</label>
        <select {...register("caseType")}>
          <option value="Corporate">Corporate</option>
          <option value="RealEstate">Real Estate</option>
          <option value="Litigation">Litigation</option>
        </select>
        {errors.caseType && <span className="text-red-500">
{errors.caseType.message}</span>}
      </div>

      <Button type="submit" disabled={createMutation.isPending}>
        {createMutation.isPending ? "Creating..." : "Create Case"}
      </Button>
    </form>
  );
}
```

List Component with Pagination

```
import { useState } from "react";
import { trpc } from "@/lib/trpc";
import { Button } from "@/components/ui/button";

export function CaseList() {
  const [page, setPage] = useState(1);
  const pageSize = 10;

  const { data, isLoading } = trpc.cases.list.useQuery({
    limit: pageSize,
    offset: (page - 1) * pageSize,
  });

  return (
    <div>
      <table>
        <thead>
          <tr>
            <th>Case ID</th>
            <th>Client</th>
            <th>Type</th>
            <th>Status</th>
          </tr>
        </thead>
        <tbody>
          {data?.cases.map((case_) => (
            <tr key={case_.id}>
              <td>{case_.id}</td>
              <td>{case_.client.name}</td>
              <td>{case_.caseType}</td>
              <td>{case_.status}</td>
            </tr>
          ))}
        </tbody>
      </table>

      <div className="flex gap-2 mt-4">
        <Button
          onClick={() => setPage(p => Math.max(1, p - 1))}
          disabled={page === 1}
        >
          Previous
        </Button>
```

```
        <span>Page {page}</span>
        <Button
          onClick={() => setPage(p => p + 1)}
          disabled={!data?.hasMore}
        >
          Next
        </Button>
      </div>
    </div>
  );
}
```

Error Handling

Backend Error Handling

```
import { TRPCError } from "@trpc/server";

export const appRouter = router({
  cases: router({
    getById: protectedProcedure
      .input(z.object({ id: z.number() }))
      .query(async ({ input }) => {
        const caseData = await db.getCaseById(input.id);

        if (!caseData) {
          throw new TRPCError({
            code: "NOT_FOUND",
            message: `Case with ID ${input.id} not found`,
          });
        }

        return caseData;
      }),

    update: protectedProcedure
      .input(z.object({ id: z.number(), status: z.string() }))
      .mutation(async ({ ctx, input }) => {
        const caseData = await db.getCaseById(input.id);

        if (!caseData) {
          throw new TRPCError({
            code: "NOT_FOUND",
            message: "Case not found",
          });
        }

        // Check authorization
        if (!caseData.assignedTeam.includes(ctx.user.id) && ctx.user.role
!== "admin") {
          throw new TRPCError({
            code: "FORBIDDEN",
            message: "You don't have permission to update this case",
          });
        }
```

```
        try {
          return await db.updateCase(input.id, { status: input.status });
        } catch (error) {
          throw new TRPCError({
            code: "INTERNAL_SERVER_ERROR",
            message: "Failed to update case",
          });
        }
      }),
  }),
});
```

Frontend Error Handling

```
import { toast } from "sonner";

export function CaseForm() {
  const createMutation = trpc.cases.create.useMutation({
    onSuccess: (data) => {
      toast.success("Case created successfully");
      // Navigate or refresh
    },
    onError: (error) => {
      // Handle specific error codes
      if (error.data?.code === "FORBIDDEN") {
        toast.error("You don't have permission to create cases");
      } else if (error.data?.code === "BAD_REQUEST") {
        toast.error("Invalid input provided");
      } else {
        toast.error(error.message || "An error occurred");
      }
    },
  });

  return (
    <form onSubmit={handleSubmit(async (data) => {
      try {
        await createMutation.mutateAsync(data);
      } catch (error) {
        // Error is already handled by onError
      }
    })}>
      {/* Form fields */}
    </form>
  );
}
```

Testing Patterns

Unit Test Example

```
import { describe, it, expect, beforeEach } from "vitest";
import { db } from "./db";

describe("Case Database Operations", () => {
  beforeEach(async () => {
    // Setup: Create test data
    await db.insert(clients).values({
      id: 1,
      name: "Test Client",
      email: "test@example.com",
    });
  });

  it("should create a case", async () => {
    const result = await db.createCase({
      clientId: 1,
      caseType: "Corporate",
      status: "Intake",
    });

    expect(result).toBeDefined();
    expect(result.clientId).toBe(1);
    expect(result.status).toBe("Intake");
  });

  it("should get case by ID", async () => {
    const created = await db.createCase({
      clientId: 1,
      caseType: "Corporate",
      status: "Intake",
    });

    const retrieved = await db.getCaseById(created.id);

    expect(retrieved).toBeDefined();
    expect(retrieved?.id).toBe(created.id);
  });

  it("should update case status", async () => {
    const created = await db.createCase({
```

```
      clientId: 1,
      caseType: "Corporate",
      status: "Intake",
    });

    const updated = await db.updateCase(created.id, { status: "Active" });

    expect(updated.status).toBe("Active");
  });
});
```

Integration Test Example

```
import { describe, it, expect } from "vitest";
import { appRouter } from "./routers";

describe("Cases API Integration", () => {
  it("should create and retrieve a case", async () => {
    const ctx = createTestContext();
    const caller = appRouter.createCaller(ctx);

    // Create case
    const created = await caller.cases.create({
      clientId: 1,
      caseType: "Corporate",
      deadline: new Date(),
    });

    expect(created).toBeDefined();

    // Retrieve case
    const retrieved = await caller.cases.getById({ id: created.id });

    expect(retrieved.id).toBe(created.id);
    expect(retrieved.caseType).toBe("Corporate");
  });
});
```

AI Agent Integration

Calling an AI Agent

```
import { invokeLLM } from "./server/_core/llm";

export async function generateCasePlan(caseData: CaseInput) {
  const prompt = `
    You are a legal case planner. Create a detailed task roadmap for:
    - Case Type: ${caseData.caseType}
    - Client: ${caseData.clientName}
    - Deadline: ${caseData.deadline}

    Return a JSON object with phases and tasks.
  `;

  const response = await invokeLLM({
    messages: [
      { role: "system", content: "You are a legal case planning expert." },
      { role: "user", content: prompt },
    ],
    response_format: {
      type: "json_schema",
      json_schema: {
        name: "case_plan",
        strict: true,
        schema: {
          type: "object",
          properties: {
            phases: {
              type: "array",
              items: {
                type: "object",
                properties: {
                  name: { type: "string" },
                  tasks: {
                    type: "array",
                    items: {
                      type: "object",
                      properties: {
                        title: { type: "string" },
                        estimatedHours: { type: "number" },
                      },
                    },
```

```
                    },
                  },
                },
              },
            },
          },
        },
      },
    },
  });

  return JSON.parse(response.choices[0].message.content);
}
```

Common Pitfalls

Pitfall 1: Infinite Query Loops

Problem:

```
// ❌ BAD: New object every render
const { data } = trpc.cases.list.useQuery({
  filter: { status: "Active" }, // New object every render!
});
```

Solution:

```
// ✅ GOOD: Stable reference
const filter = useMemo(() => ({ status: "Active" }), []);
const { data } = trpc.cases.list.useQuery(filter);
```

Pitfall 2: Not Handling Loading States

Problem:

```
// ❌ BAD: No loading state
const { data } = trpc.cases.list.useQuery();
return <div>{data?.map(...)}</div>; // Crashes if data is undefined
```

Solution:

```
// ✅ GOOD: Handle loading state
const { data, isLoading } = trpc.cases.list.useQuery();
if (isLoading) return <Loader />;
return <div>{data?.map(...)}</div>;
```

Pitfall 3: Missing Error Boundaries

Problem:

```
// ❌ BAD: No error handling
const createMutation = trpc.cases.create.useMutation();
await createMutation.mutateAsync(data); // Could throw!
```

Solution:

```
// ✅ GOOD: Handle errors
const createMutation = trpc.cases.create.useMutation({
  onError: (error) => {
    toast.error(error.message);
  },
});
await createMutation.mutateAsync(data);
```

Pitfall 4: Not Validating Input

Problem:

```
// ❌ BAD: No validation
export const createCase = protectedProcedure
  .input(z.any()) // Dangerous!
  .mutation(async ({ input }) => {
    return db.createCase(input);
  });
```

Solution:

```
// ✅ GOOD: Strict validation
export const createCase = protectedProcedure
  .input(z.object({
    clientId: z.number().positive(),
    caseType: z.enum(["Corporate", "RealEstate", "Litigation"]),
    deadline: z.date().min(new Date()),
  }))
  .mutation(async ({ input }) => {
    return db.createCase(input);
  });
```

Pitfall 5: Storing Sensitive Data in Frontend

Problem:

```
// ❌ BAD: Storing API keys in frontend
const API_KEY = "sk-xxx"; // Exposed!
```

Solution:

```
// ✅ GOOD: Use backend for sensitive operations
// Frontend calls backend procedure
const result = await trpc.ai.generatePlan.mutate(data);
// Backend handles API calls securely
```

Best Practices Summary

1. **Type Safety:** Always use TypeScript and Zod for validation
2. **Error Handling:** Handle all error cases with meaningful messages
3. **Loading States:** Show loading indicators for async operations
4. **Optimistic Updates:** Update UI immediately for better UX
5. **Testing:** Write tests for critical functionality
6. **Documentation:** Document complex logic and edge cases
7. **Performance:** Use memoization and lazy loading
8. **Security:** Never expose secrets, validate all inputs
9. **Accessibility:** Use semantic HTML and ARIA labels
10. **Code Organization:** Keep files small and focused

Document Version: 1.0
Last Updated: March 12, 2026
Maintained By: Development Team